

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
diabetes_df = pd.read_csv("diabetes.csv")

print("First 5 rows of the dataset:")
print(diabetes_df.head())
print("\nDataset Information:")
print(diabetes_df.info())
print("\nStatistical Summary:")
print(diabetes_df.describe())
print("\nMissing Values in Each Column:")
print(diabetes_df.isnull().sum())
plt.figure(figsize=(6,4))
sns.countplot(x='Outcome', data=diabetes_df)
plt.title('Distribution of Diabetes Outcome')
plt.xlabel('Outcome (0 = Non-Diabetic, 1 = Diabetic)')
plt.ylabel('Count')
plt.show()

plt.figure(figsize=(10,8))
sns.heatmap(diabetes_df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap of Features')
plt.show()
```

First 5 rows of the dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 768 entries, 0 to 767

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)

memory usage: 54.1 KB

None

Statistical Summary:

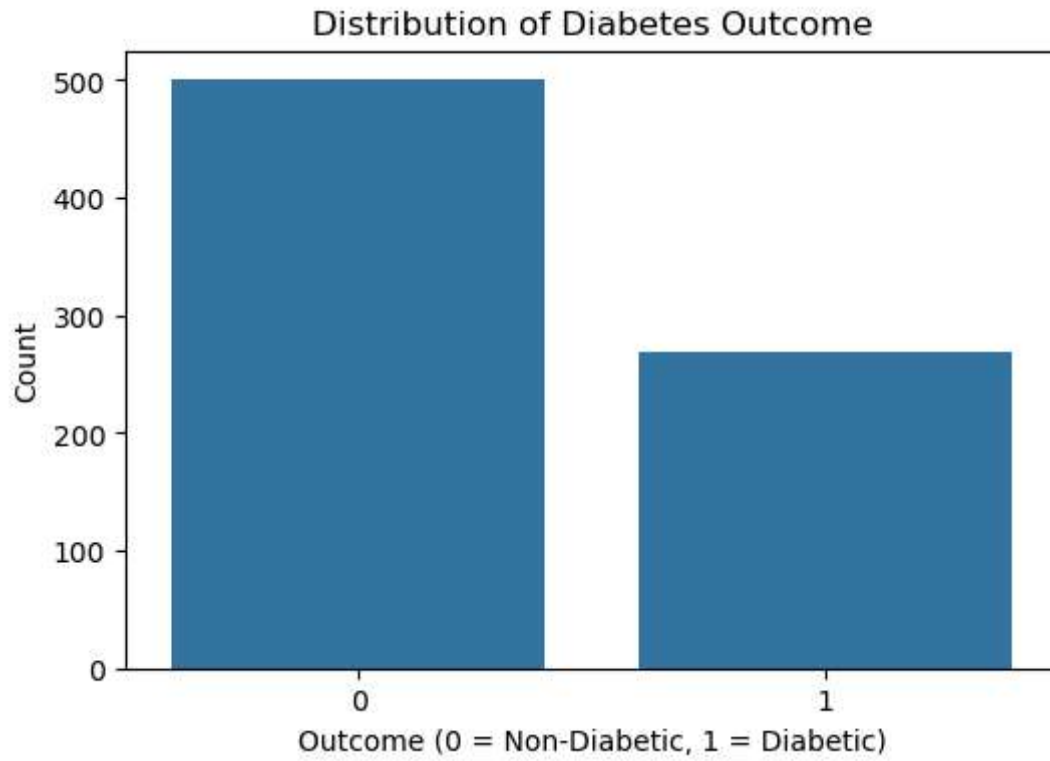
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin \
count	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479
std	3.369578	31.972618	19.355807	15.952218	115.244002
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000
75%	6.000000	140.250000	80.000000	32.000000	127.250000
max	17.000000	199.000000	122.000000	99.000000	846.000000

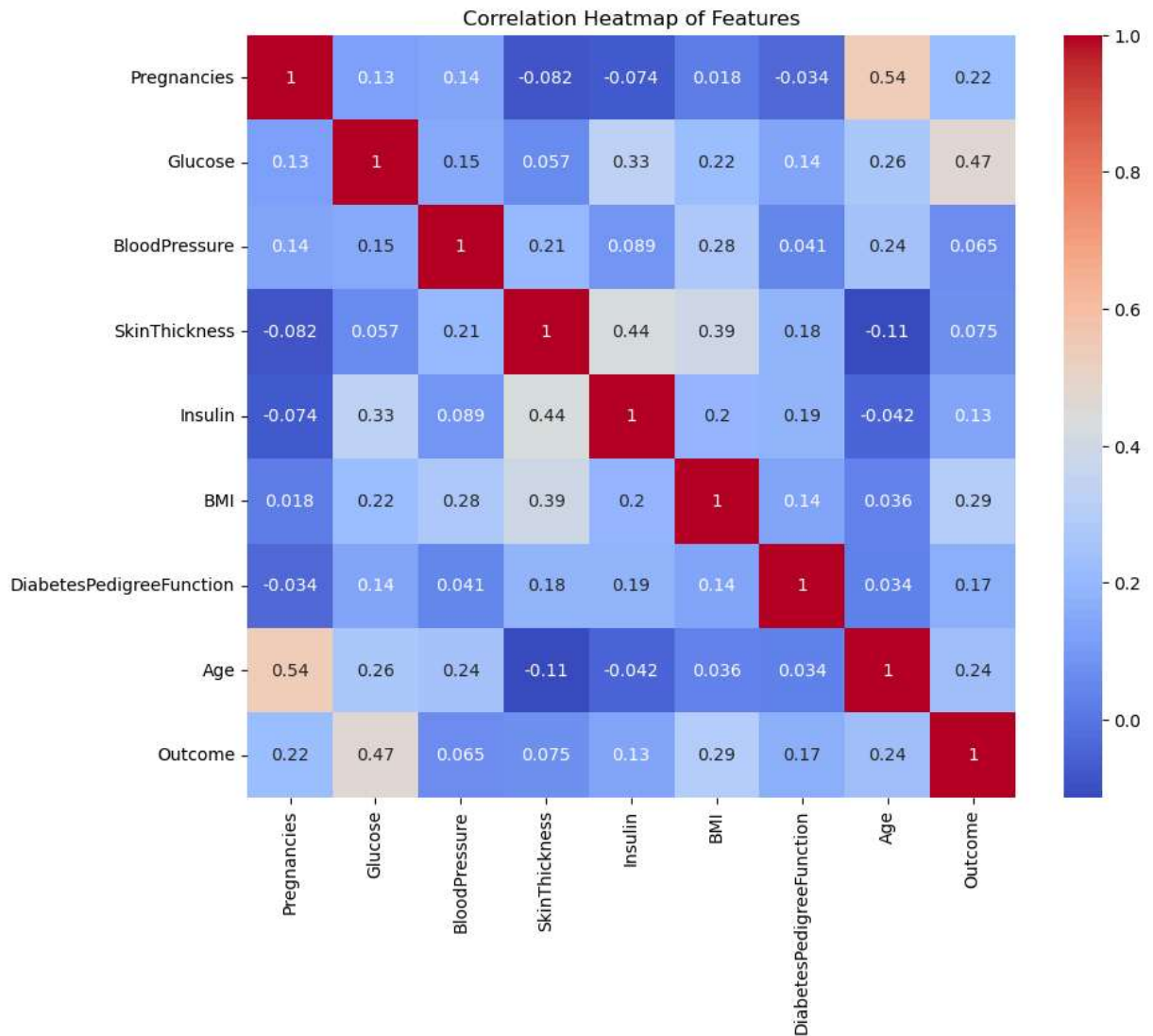
	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000
mean	31.992578	0.471876	33.240885	0.348958
std	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.078000	21.000000	0.000000
25%	27.300000	0.243750	24.000000	0.000000
50%	32.000000	0.372500	29.000000	0.000000
75%	36.600000	0.626250	41.000000	1.000000
max	67.100000	2.420000	81.000000	1.000000

Missing Values in Each Column:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64





```
In [15]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
from scipy import stats

if not os.path.exists('website_traffic.csv'):
    print("Creating sample dataset 'website_traffic.csv'...")
    dates = pd.date_range(start='2023-01-01', end='2023-12-31', freq='D')
    n = len(dates)
    page_views = np.random.randint(1000, 5000, n) + 300 * np.sin(2 * np.pi * dates)
    unique_visitors = page_views * np.random.uniform(0.4, 0.9, n)
    bounce_rate = np.random.uniform(20, 80, n)
    df_sample = pd.DataFrame({
        'Date': dates,
        'Page_Views': page_views.astype(int),
        'Unique_Visitors': unique_visitors.astype(int),
        'Bounce_Rate': bounce_rate.round(2)
    })
    df_sample.to_csv('website_traffic.csv', index=False)
```

```

df = pd.read_csv('website_traffic.csv', parse_dates=['Date'])

df = df.sort_values('Date')

df.set_index('Date', inplace=True)

df = df.interpolate()

print("First few rows of the dataset:")
print(df.head())
print("\nDataset Information:")
print(df.info())

decomposition = seasonal_decompose(df['Page_Views'], model='additive', period=7) #

plt.figure(figsize=(12, 8))
decomposition.plot()
plt.suptitle('Time Series Decomposition of Page Views', fontsize=14)
plt.show()

plt.figure(figsize=(14, 6))
plt.plot(df.index, df['Page_Views'], label='Page Views')
plt.plot(df.index, df['Unique_Visitors'], label='Unique Visitors')
plt.plot(df.index, df['Bounce_Rate'], label='Bounce Rate')
plt.title('Website Traffic Metrics Over Time')
plt.xlabel('Date')
plt.ylabel('Values')
plt.legend()
plt.show()

df['PageViews_MA7'] = df['Page_Views'].rolling(window=7).mean()
plt.figure(figsize=(14, 5))
plt.plot(df.index, df['Page_Views'], label='Original')
plt.plot(df.index, df['PageViews_MA7'], label='7-Day Moving Average', linewidth=3)
plt.title('Page Views with Moving Average')
plt.legend()
plt.show()

df['DayOfWeek'] = df.index.day_name()
plt.figure(figsize=(10, 5))
sns.boxplot(
    x='DayOfWeek',
    y='Page_Views',
    data=df.reset_index(),
    order=['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
)
plt.title('Seasonal Pattern by Day of Week')
plt.xticks(rotation=45)
plt.show()

# Step 5: Anomaly Detection using Z-score
df['Z_Score'] = np.abs(stats.zscore(df['Page_Views']))

# Mark anomalies (Z-score > 3)
anomalies = df[df['Z_Score'] > 3]

```

```
plt.figure(figsize=(14, 5))
plt.plot(df.index, df['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='Anomalies')
plt.title('Anomaly Detection in Page Views')
plt.legend()
plt.show()

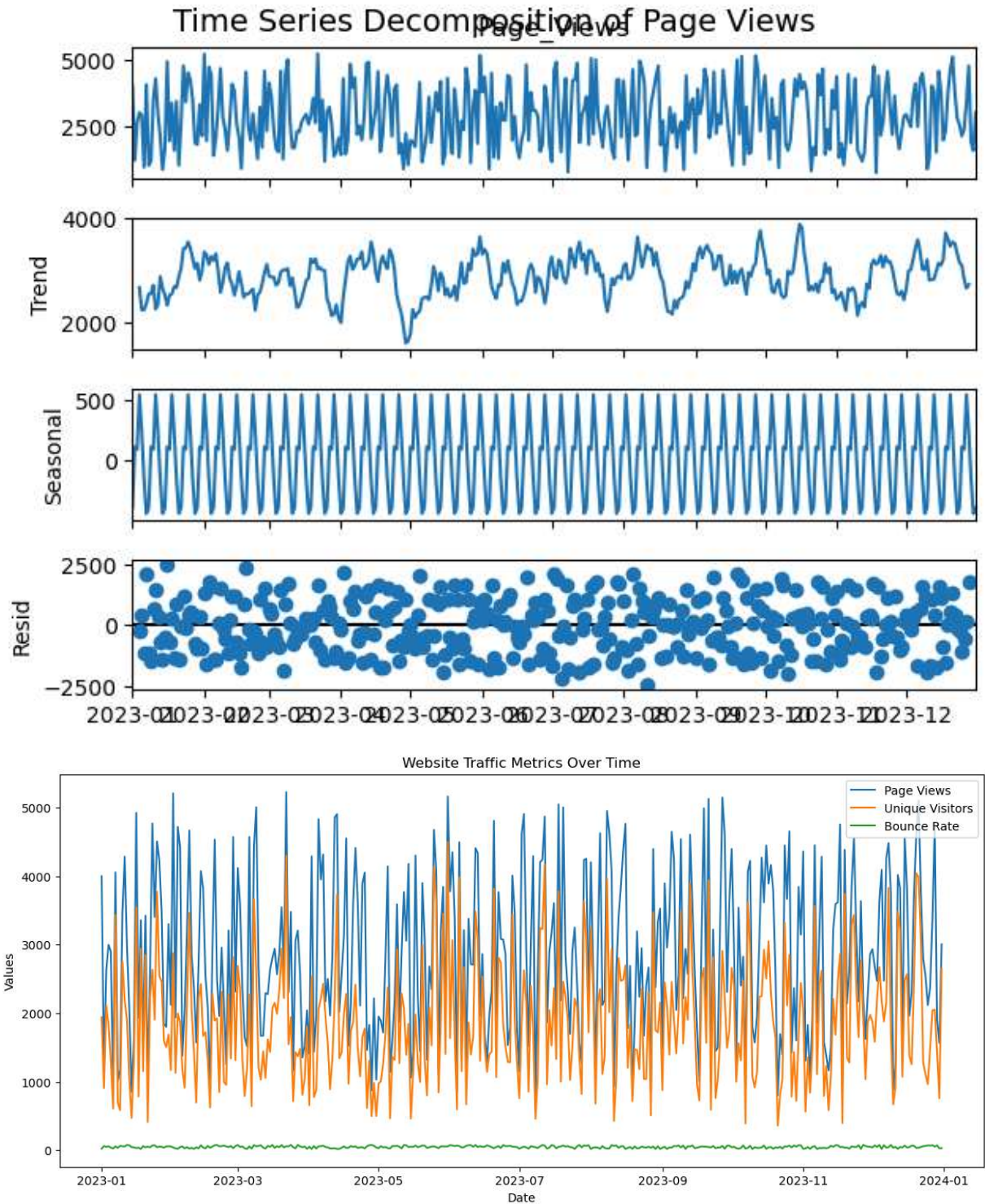
print("\nDetected Anomalies:")
print(anomalies[['Page_Views', 'Z_Score']])
```

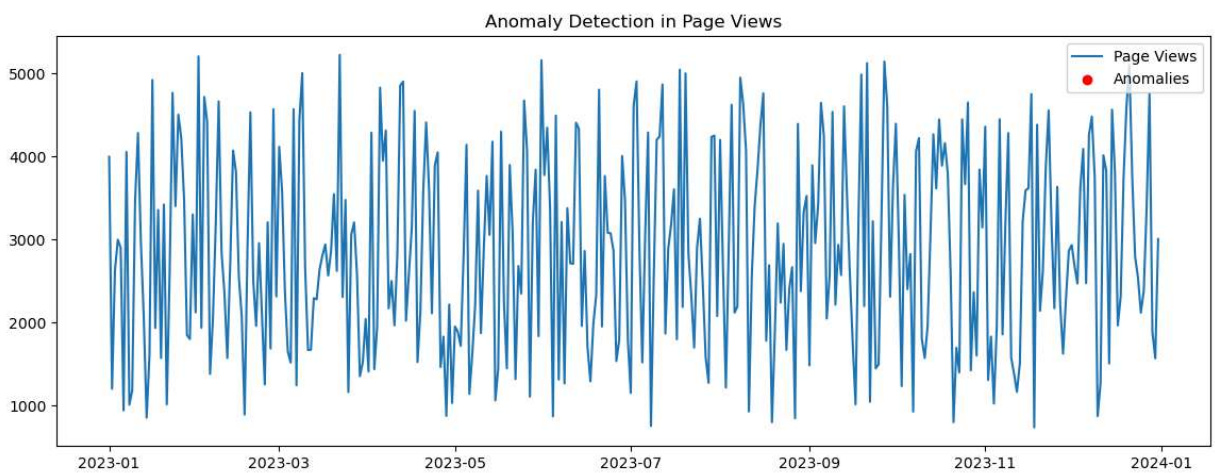
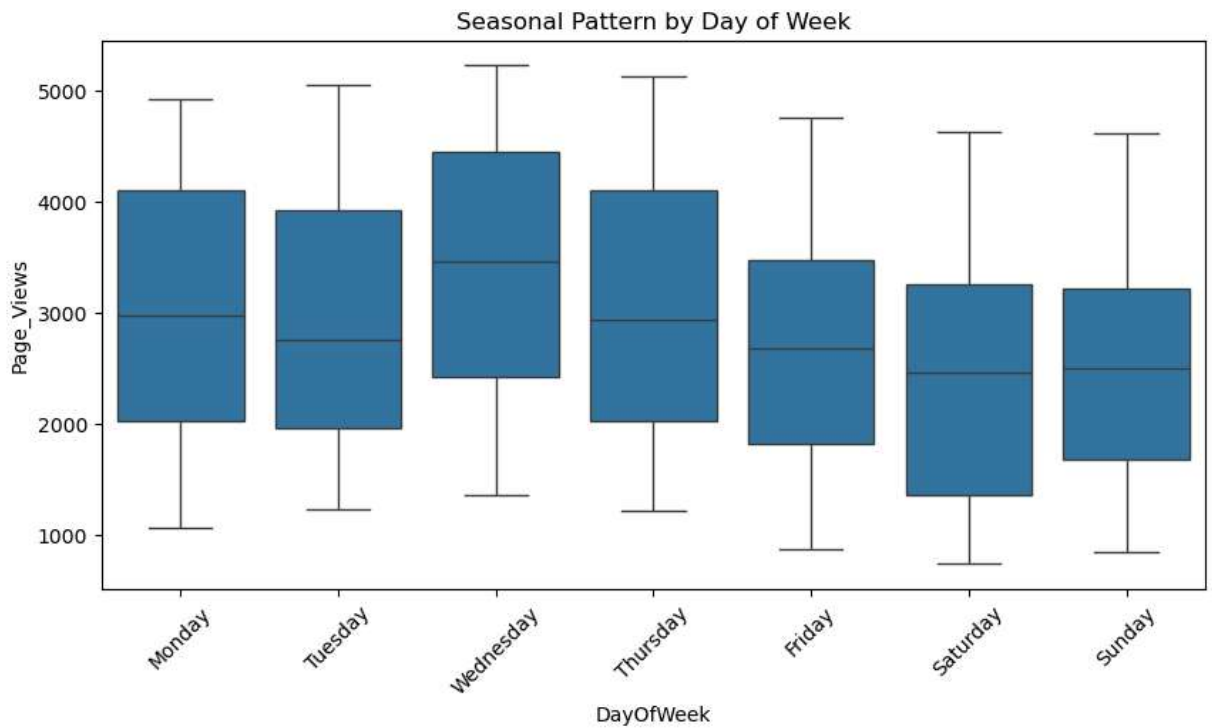
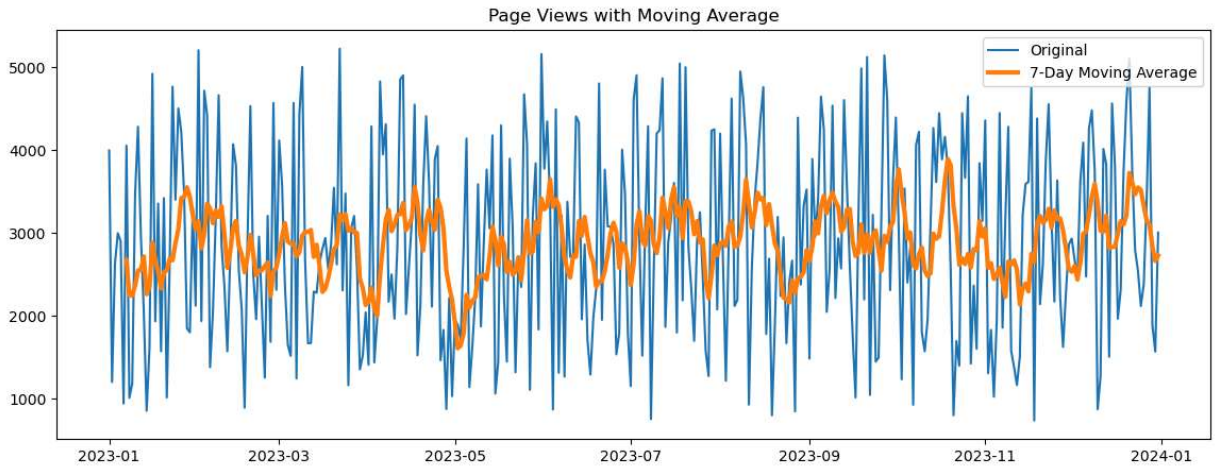
First few rows of the dataset:

	Page_Views	Unique_Visitors	Bounce_Rate
Date			
2023-01-01	3999	1942	25.20
2023-01-02	1202	908	62.76
2023-01-03	2627	2112	51.91
2023-01-04	2999	1833	59.74
2023-01-05	2901	1316	39.68

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 365 entries, 2023-01-01 to 2023-12-31
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Page_Views      365 non-null   int64
1   Unique_Visitors 365 non-null   int64
2   Bounce_Rate     365 non-null   float64
dtypes: float64(1), int64(2)
memory usage: 11.4 KB
None
<Figure size 1200x800 with 0 Axes>
```





Detected Anomalies:
Empty DataFrame
Columns: [Page_Views, Z_Score]
Index: []


```

In [10]: import numpy as np
import matplotlib.pyplot as plt

population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(population_mean, population_std, population_size)

print(f"Population Mean: {np.mean(population):.2f}")
print(f"Population Std Dev: {np.std(population):.2f}")

plt.figure(figsize=(8,4))
plt.hist(population, bins=50, color='skyblue', edgecolor='black', alpha=0.7)
plt.title("Population Distribution (Normal)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()

sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
    means = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        means.append(np.mean(sample))
    sample_means[size] = means

plt.figure(figsize=(12, 8))

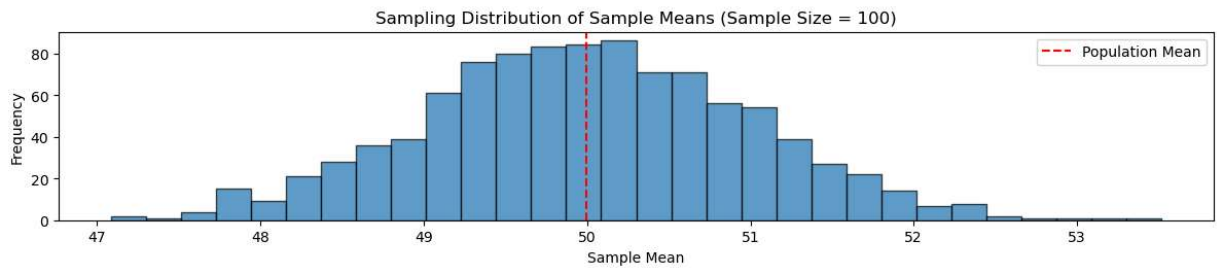
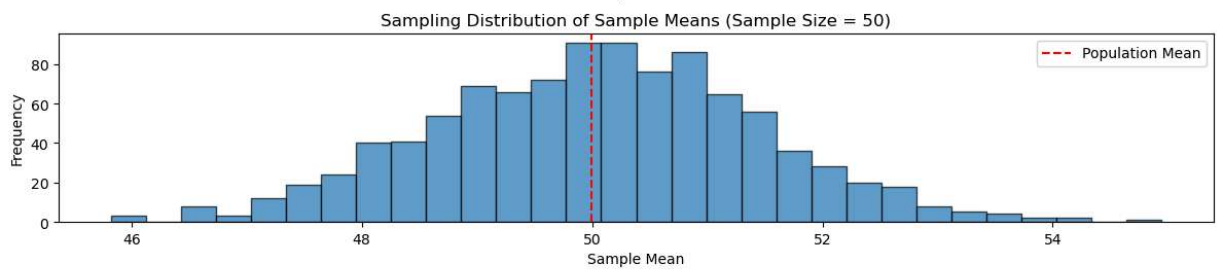
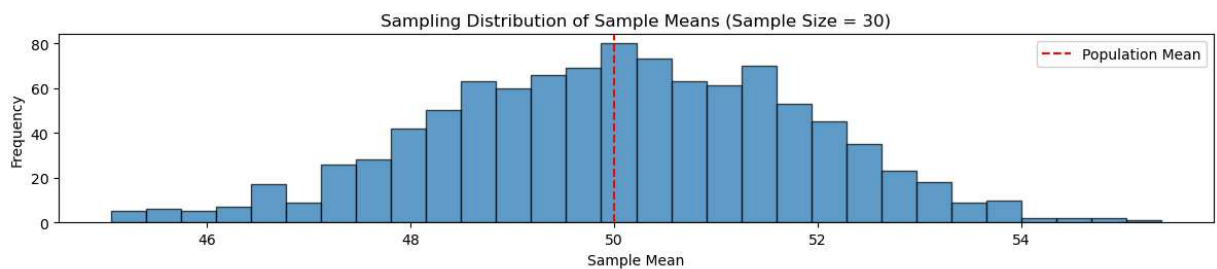
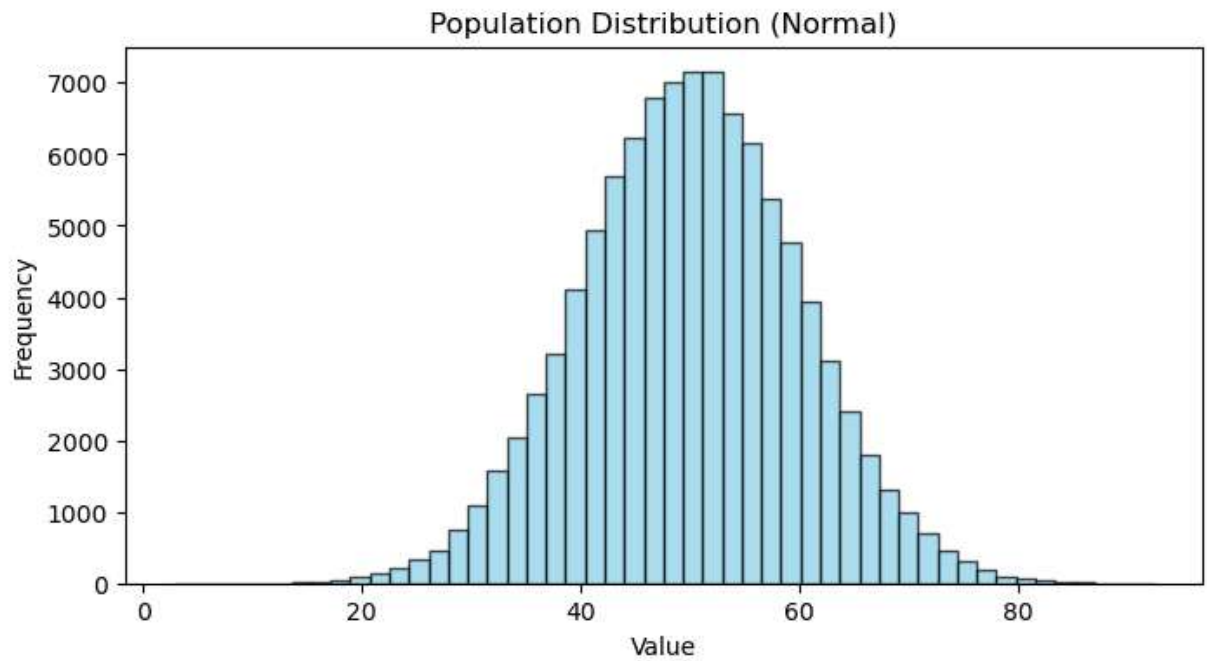
for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i+1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, edgecolor='black')
    plt.axvline(np.mean(population), color='red', linestyle='dashed', linewidth=1.5)
    plt.title(f"Sampling Distribution of Sample Means (Sample Size = {size})")
    plt.xlabel("Sample Mean")
    plt.ylabel("Frequency")
    plt.legend()

plt.tight_layout()
plt.show()

print("\nCentral Limit Theorem Demonstration:")
for size in sample_sizes:
    print(f"Sample Size {size}: Mean of Sample Means = {np.mean(sample_means[size])}")

```

Population Mean: 50.00
Population Std Dev: 9.97



Central Limit Theorem Demonstration:

Sample Size 30: Mean of Sample Means = 50.05, Std Dev = 1.77

Sample Size 50: Mean of Sample Means = 50.03, Std Dev = 1.40

Sample Size 100: Mean of Sample Means = 50.00, Std Dev = 1.00